30. *SINGLE LINKED LIST – MEMORY REFERENCING AND FREEING MEMORY*

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;
```

***What the above code depicts*: –**

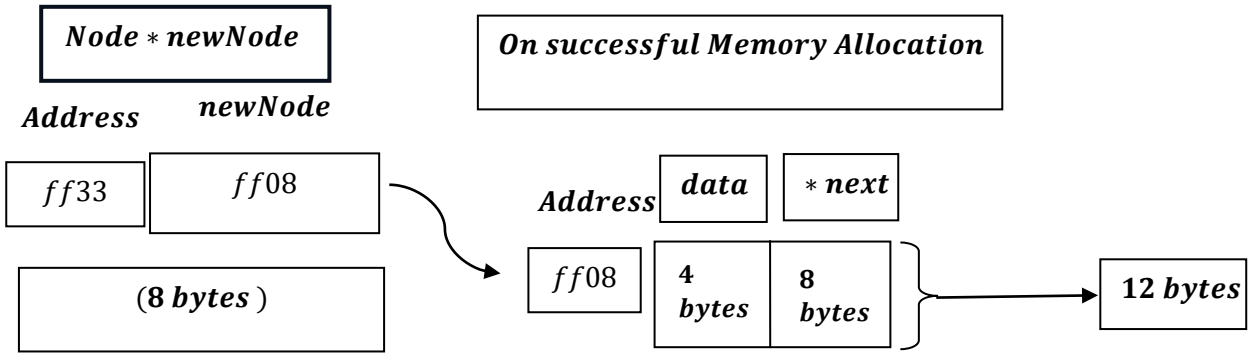
→ ***struct whose name is ListNode is renamed with Node through `typedef` keyword.***

→ ***for struct Node , there will be a memory allocation for integer variable `data`.***

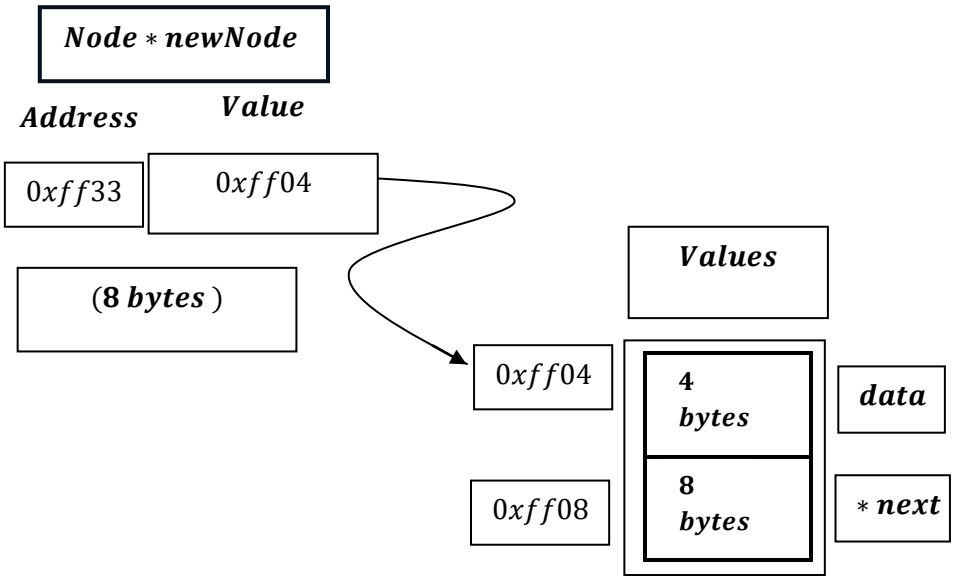
→ ***for struct Node, there will be a memory allocation for pointer next.***

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

***We know*[*Based on 64 – bit system*]:**



But memory allocation doesn't take like this, actually, this is for understanding, and for explanation, memory allocation a little bit more complex:



At now we see newNode pointer , only pointing 0xff04 , but it should update Data part and Next pointer part, how that happens?

→ The compiler knows that the data member is at the very beginning of the struct. Its offset from the start is 0 bytes.

such as: newNode → data = 10.

$$\rightarrow baseaddress + offset = 0xff04 + 0 = 0xff04.$$

<u>The Compiler's "Secret" Calculation</u>

When the compiler sees newNode->data = 10;, it thinks:

- ***"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)***

- *"Look up the definition of the Node struct. The data member is at the beginning, so its offset is 0."*
- *"Calculate the final address: base address + offset => 0xff04 + 0 => 0xff04."*
- *"Write the integer value 10 to the memory location starting at 0xff04."*

$Next\ is * next\ pointer\ variable:$

$$say, newNode \rightarrow next = head;$$

$$\rightarrow baseaddress + offset = 0xff04 + 4\ bytes = 0xff08.$$

$$\rightarrow And\ then\ write\ head\ pointer's\ value\ to\ address\ 0xff08.$$

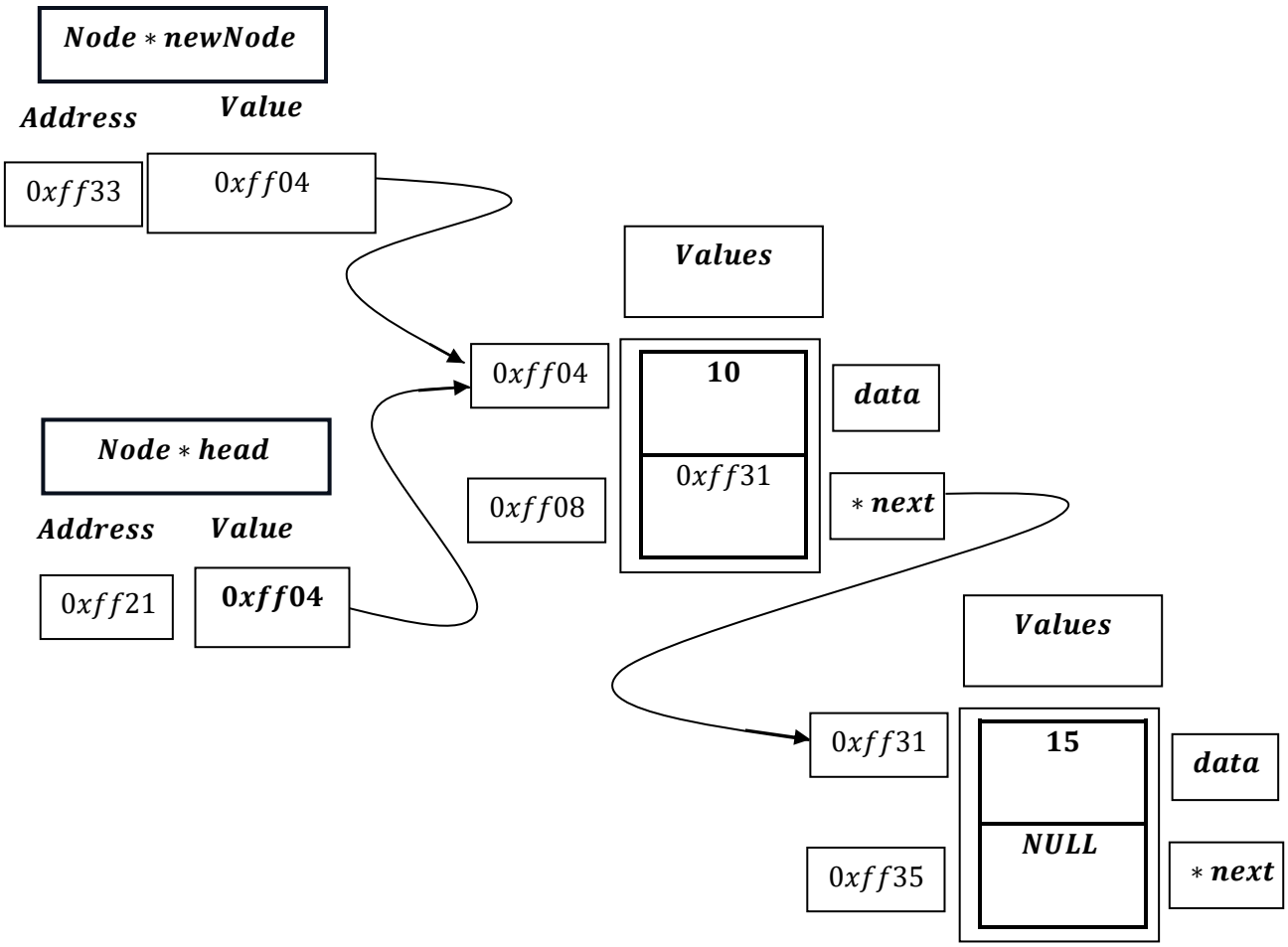
When it sees newNode->next = some_other_node;, it thinks:

1. *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
2. *"Look up the definition of the Node struct. The next member comes after data, which is 4 bytes long. So, the offset for next is 4."*
3. *"Calculate the final address: base address + offset => 0xff04 + 4 => 0xff08."*
4. *"Write the address of some_other_node to the memory location starting at 0xff08."*

$But\ this\ doesn't\ means\ , value\ of\ newNode\ pointer\ gets\ changed\ and\ it\ starts\ to\ point\ to$

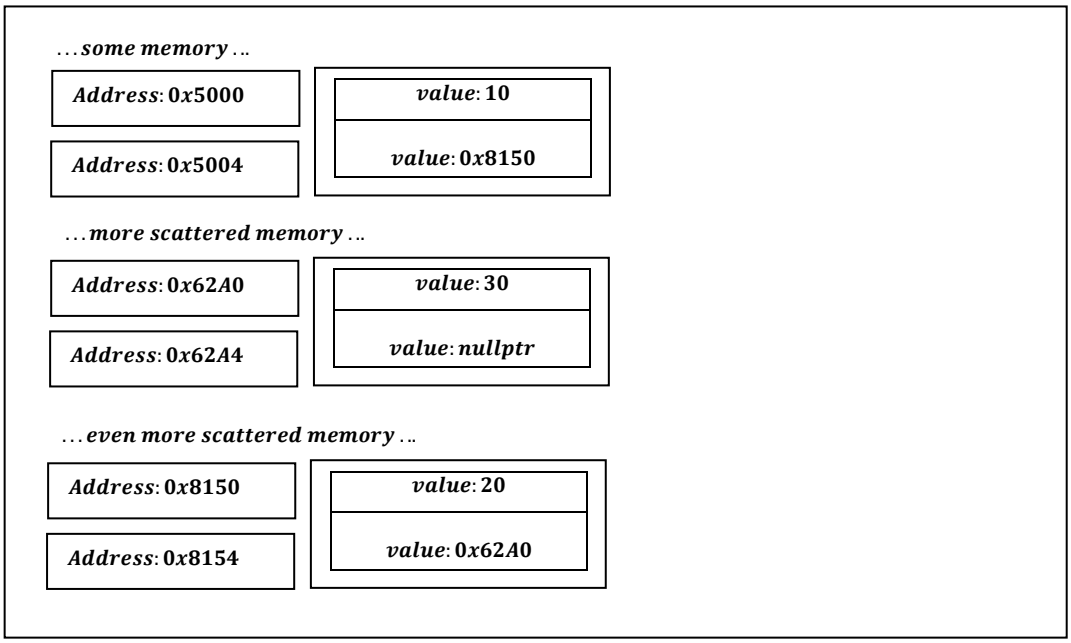
$address\ of\ next\ pointer\ variable\ .$

$So, Linked\ List\ seems\ like: -(Say, Insert\ At\ Benginning)$



***Note*: –Same process takes place for every nodes , executed by compilers .**

How does Heap memory looks like : –



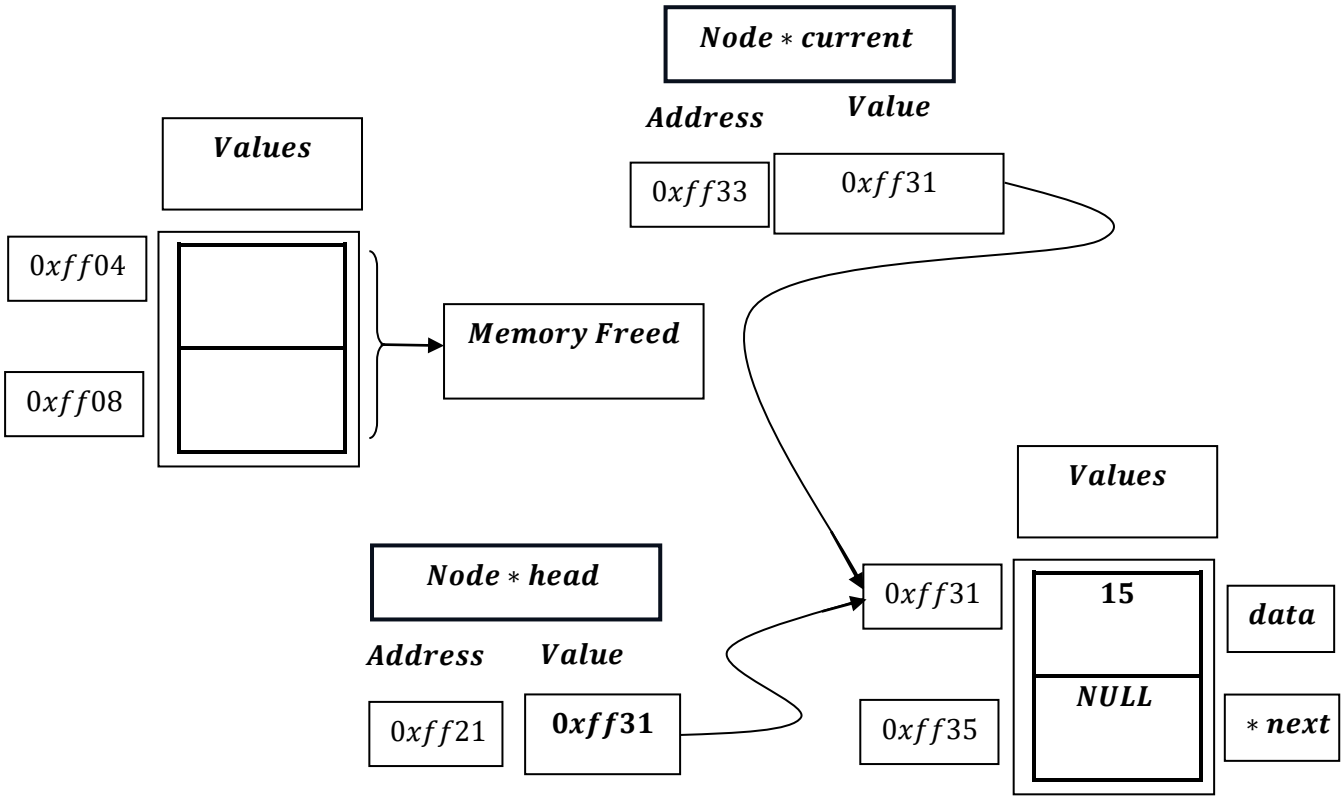
Heap Memory showing how Linked List is formed.

Freeing up the Memory

```
        free(*current);
```

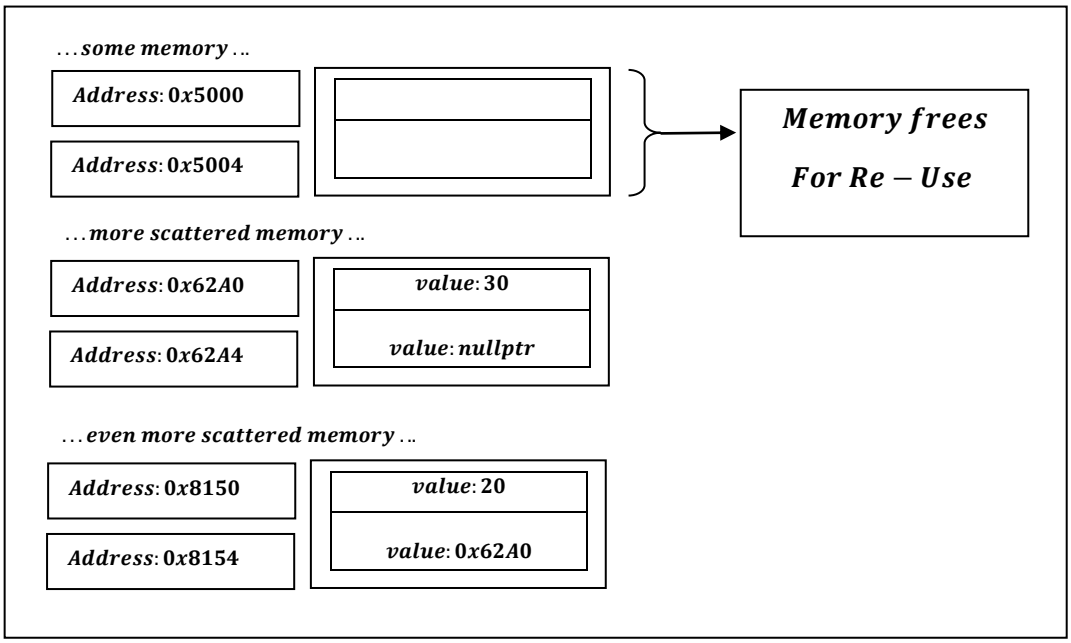
The free(current) command frees the entire memory block allocated for that node, which includes the space for both data and the next pointer.

free() doesn't know or care that the memory block is being used for a struct with different members. It only knows the starting address (current) and the size of the block that was originally allocated by malloc. It returns that entire chunk of memory to the operating system's heap for reuse.



A simple demonstration of deletion from first node.

How heap memory looks like?



Heap Memory showing how Linked List , deletion process.